



# RegEX

A quick reference for regular expressions (regex), including symbols, ranges, grouping, assertions and some sample patterns to get you started.

## # Getting Started

### Introduction

This is a quick cheat sheet to getting started with regular expressions.

- [Regex in Python](#) (quickref.me)
- [Regex in JavaScript](#) (quickref.me)
- [Regex in PHP](#) (quickref.me)
- [Regex in Java](#) (quickref.me)
- [Regex in MySQL](#) (quickref.me)
- [Regex in Vim](#) (quickref.me)
- [Regex in Emacs](#) (quickref.me)
- [Online regex tester](#) (regex101.com)

### Character Classes

|                       |                                         |
|-----------------------|-----------------------------------------|
| <code>[abc]</code>    | A single character of: a, b or c        |
| <code>[^abc]</code>   | A character except: a, b or c           |
| <code>[a-z]</code>    | A character in the range: a-z           |
| <code>[^a-z]</code>   | A character not in the range: a-z       |
| <code>[0-9]</code>    | A digit in the range: 0-9               |
| <code>[a-zA-Z]</code> | A character in the range:<br>a-z or A-Z |

[a-zA-Z0-9]

A character in the range:  
a-z, A-Z or 0-9

#### Quantifiers

|        |                       |
|--------|-----------------------|
| a?     | Zero or one of a      |
| a*     | Zero or more of a     |
| a+     | One or more of a      |
| [0-9]+ | One or more of 0-9    |
| a{3}   | Exactly 3 of a        |
| a{3,}  | 3 or more of a        |
| a{3,6} | Between 3 and 6 of a  |
| a*     | Greedy quantifier     |
| a*?    | Lazy quantifier       |
| a*+    | Possessive quantifier |

#### Common Metacharacters

|   |   |    |
|---|---|----|
| ^ | { | +  |
| < | [ | *  |
| ) | > | .  |
| ( |   | \$ |
| \ | ? |    |

Escape these special characters with \

#### Meta Sequences

|    |                              |
|----|------------------------------|
| .  | Any single character         |
| \s | Any whitespace character     |
| \S | Any non-whitespace character |
| \d | Any digit, Same as [0-9]     |

|                             |                                                                          |
|-----------------------------|--------------------------------------------------------------------------|
| <code>\D</code>             | Any non-digit, Same as <code>[^0-9]</code>                               |
| <code>\w</code>             | Any word character                                                       |
| <code>\W</code>             | Any non-word character                                                   |
| <code>\X</code>             | Any Unicode sequences, linebreaks included                               |
| <code>\C</code>             | Match one data unit                                                      |
| <code>\R</code>             | Unicode newlines                                                         |
| <code>\v</code>             | Vertical whitespace character                                            |
| <code>\V</code>             | Negation of <code>\v</code> - anything except newlines and vertical tabs |
| <code>\h</code>             | Horizontal whitespace character                                          |
| <code>\H</code>             | Negation of <code>\h</code>                                              |
| <code>\K</code>             | Reset match                                                              |
| <code>\n</code>             | Match nth subpattern                                                     |
| <code>\pX</code>            | Unicode property X                                                       |
| <code>\p{...}</code>        | Unicode property or script category                                      |
| <code>\PX</code>            | Negation of <code>\pX</code>                                             |
| <code>\P{...}</code>        | Negation of <code>\p</code>                                              |
| <code>\Q...\E</code>        | Quote; treat as literals                                                 |
| <code>\k&lt;name&gt;</code> | Match subpattern <code>name</code>                                       |
| <code>\k' name '</code>     | Match subpattern <code>name</code>                                       |
| <code>\k{name}</code>       | Match subpattern <code>name</code>                                       |
| <code>\gn</code>            | Match nth subpattern                                                     |
| <code>\g{n}</code>          | Match nth subpattern                                                     |
| <code>\g&lt;n&gt;</code>    | Recurse nth capture group                                                |
| <code>\g'n'</code>          | Recurse nth capture group.                                               |

|                               |                                                    |
|-------------------------------|----------------------------------------------------|
| <code>\g{-n}</code>           | Match nth relative previous subpattern             |
| <code>\g{&lt;+n&gt;}</code>   | Recurse nth relative upcoming subpattern           |
| <code>\g'+n'</code>           | Match nth relative upcoming subpattern             |
| <code>\g'letter'</code>       | Recurse named capture group <b>letter</b>          |
| <code>\g{letter}</code>       | Match previously-named capture group <b>letter</b> |
| <code>\g&lt;letter&gt;</code> | Recurse named capture group <b>letter</b>          |
| <code>\xYY</code>             | Hex character YY                                   |
| <code>\x{YYYY}</code>         | Hex character YYYY                                 |
| <code>\ddd</code>             | Octal character ddd                                |
| <code>\cY</code>              | Control character Y                                |
| <code>[\b]</code>             | Backspace character                                |
| <code>\</code>                | Makes any character literal                        |

#### Anchors

|                 |                        |
|-----------------|------------------------|
| <code>\G</code> | Start of match         |
| <code>^</code>  | Start of string        |
| <code>\$</code> | End of string          |
| <code>\A</code> | Start of string        |
| <code>\Z</code> | End of string          |
| <code>\z</code> | Absolute end of string |
| <code>\b</code> | A word boundary        |
| <code>\B</code> | Non-word boundary      |

#### Substitution

|                 |                             |
|-----------------|-----------------------------|
| <code>\0</code> | Complete match contents     |
| <code>\1</code> | Contents in capture group 1 |

|                       |                                            |
|-----------------------|--------------------------------------------|
| <code>\$1</code>      | Contents in capture group 1                |
| <code>\${foo}</code>  | Contents in capture group <code>foo</code> |
| <code>\x20</code>     | Hexadecimal replacement values             |
| <code>\x{06fa}</code> | Hexadecimal replacement values             |
| <code>\t</code>       | Tab                                        |
| <code>\r</code>       | Carriage return                            |
| <code>\n</code>       | Newline                                    |
| <code>\f</code>       | Form-feed                                  |
| <code>\U</code>       | Uppercase Transformation                   |
| <code>\L</code>       | Lowercase Transformation                   |
| <code>\E</code>       | Terminate any Transformation               |

#### Group Constructs

|                                  |                                       |
|----------------------------------|---------------------------------------|
| <code>(...)</code>               | Capture everything enclosed           |
| <code>(a b)</code>               | Match either a or b                   |
| <code>(?:...)</code>             | Match everything enclosed             |
| <code>(?&gt;...)</code>          | Atomic group (non-capturing)          |
| <code>(? ...)</code>             | Duplicate subpattern group number     |
| <code>(?#...)</code>             | Comment                               |
| <code>(?'name'...)</code>        | Named Capturing Group                 |
| <code>(?&lt;name&gt;...)</code>  | Named Capturing Group                 |
| <code>(?P&lt;name&gt;...)</code> | Named Capturing Group                 |
| <code>(?imsxXU)</code>           | Inline modifiers                      |
| <code>(?(DEFINE)...) </code>     | Pre-define patterns before using them |

#### Assertions

|                                    |                                 |
|------------------------------------|---------------------------------|
| <code>(?(1)yes no)</code>          | Conditional statement           |
| <code>(?(R)yes no)</code>          | Conditional statement           |
| <code>(?(R#)yes no)</code>         | Recursive Conditional statement |
| <code>(?(R&amp;name)yes no)</code> | Conditional statement           |
| <code>(?(?=...)yes no)</code>      | Lookahead conditional           |
| <code>(?(?&lt;=...)yes no)</code>  | Lookbehind conditional          |

#### Lookarounds

|                          |                     |
|--------------------------|---------------------|
| <code>(?=...)</code>     | Positive Lookahead  |
| <code>(?!...)</code>     | Negative Lookahead  |
| <code>(?&lt;=...)</code> | Positive Lookbehind |
| <code>(?&lt;!...)</code> | Negative Lookbehind |

Lookaround lets you match a group before (lookbehind) or after (lookahead) your main pattern without including it in the result.

#### Flags/Modifiers

|                |                       |
|----------------|-----------------------|
| <code>g</code> | Global                |
| <code>m</code> | Multiline             |
| <code>i</code> | Case insensitive      |
| <code>x</code> | Ignore whitespace     |
| <code>s</code> | Single line           |
| <code>u</code> | Unicode               |
| <code>X</code> | eXtended              |
| <code>U</code> | Ungreedy              |
| <code>A</code> | Anchor                |
| <code>J</code> | Duplicate group names |

#### Recurse

|           |                                   |
|-----------|-----------------------------------|
| (?R)      | Recurse entire pattern            |
| (?1)      | Recurse first subpattern          |
| (?+1)     | Recurse first relative subpattern |
| (?&name)  | Recurse subpattern <b>name</b>    |
| (?P=name) | Match subpattern <b>name</b>      |
| (?P>name) | Recurse subpattern <b>name</b>    |

## POSIX Character Classes

| Character Class           | Same as                                                    | Meaning                        |
|---------------------------|------------------------------------------------------------|--------------------------------|
| <code>[[:alnum:]]</code>  | <code>[0-9A-Za-z]</code>                                   | Letters and digits             |
| <code>[[:alpha:]]</code>  | <code>[A-Za-z]</code>                                      | Letters                        |
| <code>[[:ascii:]]</code>  | <code>[\x00-\x7F]</code>                                   | ASCII codes 0-127              |
| <code>[[:blank:]]</code>  | <code>[\t ]</code>                                         | Space or tab only              |
| <code>[[:cntrl:]]</code>  | <code>[\x00-\x1F\x7F]</code>                               | Control characters             |
| <code>[[:digit:]]</code>  | <code>[0-9]</code>                                         | Decimal digits                 |
| <code>[[:graph:]]</code>  | <code>[[:alnum:]][[:punct:]]</code>                        | Visible characters (not space) |
| <code>[[:lower:]]</code>  | <code>[a-z]</code>                                         | Lowercase letters              |
| <code>[[:print:]]</code>  | <code>[ -~] == [[:graph:]]</code>                          | Visible characters             |
| <code>[[:punct:]]</code>  | <code>[!"#\$%&amp;'()*+,-./:;&lt;=&gt;?@[ ]^_`{ }~]</code> | Visible punctuation characters |
| <code>[[:space:]]</code>  | <code>[\t\n\v\f\r ]</code>                                 | Whitespace                     |
| <code>[[:upper:]]</code>  | <code>[A-Z]</code>                                         | Uppercase letters              |
| <code>[[:word:]]</code>   | <code>[0-9A-Za-z_]</code>                                  | Word characters                |
| <code>[[:xdigit:]]</code> | <code>[0-9A-Fa-f]</code>                                   | Hexadecimal digits             |
| <code>[[:&lt;:]]</code>   | <code>[\b(?:=\w)]</code>                                   | Start of word                  |
| <code>[[:&gt;:]]</code>   | <code>[\b(?:&lt;=\w)]</code>                               | End of word                    |

|                      |                       |
|----------------------|-----------------------|
| (*ACCEPT)            | Control verb          |
| (*FAIL)              | Control verb          |
| (*MARK:NAME)         | Control verb          |
| (*COMMIT)            | Control verb          |
| (*PRUNE)             | Control verb          |
| (*SKIP)              | Control verb          |
| (*THEN)              | Control verb          |
| (*UTF)               | Pattern modifier      |
| (*UTF8)              | Pattern modifier      |
| (*UTF16)             | Pattern modifier      |
| (*UTF32)             | Pattern modifier      |
| (*UCP)               | Pattern modifier      |
| (*CR)                | Line break modifier   |
| (*LF)                | Line break modifier   |
| (*CRLF)              | Line break modifier   |
| (*ANYCRLF)           | Line break modifier   |
| (*ANY)               | Line break modifier   |
| \R                   | Line break modifier   |
| (*BSR_ANYCRLF)       | Line break modifier   |
| (*BSR_UNICODE)       | Line break modifier   |
| (*LIMIT_MATCH=x)     | Regex engine modifier |
| (*LIMIT_RECURSION=d) | Regex engine modifier |
| (*NO_AUTO_POSSESS)   | Regex engine modifier |
| (*NO_START_OPT)      | Regex engine modifier |



## # Regex examples

### Characters

|             |                                                 |
|-------------|-------------------------------------------------|
| ring        | Match <b>ring</b> <b>spring</b> board etc.      |
| .           | Match <b>a</b> , <b>9</b> , <b>+</b> etc.       |
| h.o         | Match <b>hoo</b> , <b>h2o</b> , <b>h/o</b> etc. |
| ring\?      | Match <b>ring?</b>                              |
| \(quiet\)   | Match <b>(quiet)</b>                            |
| c:\\windows | Match <b>c:\windows</b>                         |

Use \ to search for these special characters:

[ \ ^ \$ . | ? \* + ( ) { }

### Alternatives

|             |                                    |
|-------------|------------------------------------|
| cat dog     | Match <b>cat</b> or <b>dog</b>     |
| id identity | Match <b>id</b> or <b>identity</b> |
| identity id | Match <b>id</b> or <b>identity</b> |

Order longer to shorter when alternatives overlap

### Character classes

|                 |                                                           |
|-----------------|-----------------------------------------------------------|
| [aeiou]         | Match any vowel                                           |
| [^aeiou]        | Match a NON vowel                                         |
| r[iau]ng        | Match <b>ring</b> , <b>wrangle</b> , <b>sprung</b> , etc. |
| gr[ae]y         | Match <b>gray</b> or <b>grey</b>                          |
| [a-zA-Z0-9]     | Match any letter or digit                                 |
| [\u3a00-\ufa99] | Match any Unicode <b>Hàn</b> (中文)                         |

In [ ] always escape . \ ] and sometimes ^ - .

### Shorthand classes

|                            |                                                    |
|----------------------------|----------------------------------------------------|
| <code>\w</code>            | "Word" character<br>(letter, digit, or underscore) |
| <code>\d</code>            | Digit                                              |
| <code>\s</code>            | Whitespace<br>(space, tab, vtab, newline)          |
| <code>\W, \D, or \S</code> | Not word, digit, or whitespace                     |
| <code>[\D\S]</code>        | Means not digit or whitespace, both match          |
| <code>[^\d\s]</code>       | Disallow digit and whitespace                      |

### Occurrences

|                                |                                                          |
|--------------------------------|----------------------------------------------------------|
| <code>colou?r</code>           | Match <b>color</b> or <b>colour</b>                      |
| <code>[BW]ill[ieamy's]*</code> | Match <b>Bill</b> , <b>Willy</b> , <b>William's</b> etc. |
| <code>[a-zA-Z]+</code>         | Match 1 or more letters                                  |
| <code>\d{3}-\d{2}-\d{4}</code> | Match a SSN                                              |
| <code>[a-z]\w{1,7}</code>      | Match a UW NetID                                         |

### Greedy versus lazy

|                                  |                                                     |
|----------------------------------|-----------------------------------------------------|
| <code>* + {n,}</code><br>greedy  | Match as much as possible                           |
| <code>&lt;.+&gt;</code>          | Finds 1 big match in <b>&lt;b&gt;bold&lt;/b&gt;</b> |
| <code>*? +? {n,}?</code><br>lazy | Match as little as possible                         |
| <code>&lt;.+?&gt;</code>         | Finds 2 matches in <b>&lt;b&gt;bold&lt;/b&gt;</b>   |

### Scope

|                     |                                             |
|---------------------|---------------------------------------------|
| <code>\b</code>     | "Word" edge (next to non "word" character)  |
| <code>\bring</code> | Word starts with "ring", ex <b>ringtone</b> |
| <code>ring\b</code> | Word ends with "ring", ex <b>spring</b>     |

|                                |                                             |
|--------------------------------|---------------------------------------------|
| <code>\b9\b</code>             | Match single digit 9, not 19, 91, 99, etc.. |
| <code>\b[a-zA-Z]{6}\b</code>   | Match 6-letter words                        |
| <code>\B</code>                | Not word edge                               |
| <code>\Bring\B</code>          | Match <b>springs</b> and <b>wringer</b>     |
| <code>^\d*\$</code>            | Entire string must be digits                |
| <code>^[a-zA-Z]{4,20}\$</code> | String must have 4-20 letters               |
| <code>^[A-Z]</code>            | String must begin with capital letter       |
| <code>[\.\!?\'"]\$</code>      | String must end with terminal punctuation   |

#### Modifiers

|                               |                                                  |
|-------------------------------|--------------------------------------------------|
| <code>(?i)[a-z]*(?-i)</code>  | Ignore case ON / OFF                             |
| <code>(?s).*?(?-s)</code>     | Match multiple lines (causes . to match newline) |
| <code>(?m)^\.*;\$(?-m)</code> | ^ & \$ match lines not whole string              |
| <code>(?x)</code>             | #free-spacing mode, this EOL comment ignored     |
| <code>(?-x)</code>            | free-spacing mode OFF                            |
| <code>/regex/ismx</code>      | Modify mode for entire string                    |

#### Groups

|                             |                                     |
|-----------------------------|-------------------------------------|
| <code>(in out)put</code>    | Match <b>input</b> or <b>output</b> |
| <code>\d{5}(-\d{4})?</code> | US zip code (" + 4" optional)       |

Parser tries EACH alternative if match fails after group.  
Can lead to catastrophic backtracking.

#### Back references

|                                     |                                                       |
|-------------------------------------|-------------------------------------------------------|
| <code>(to) (be) or not \1 \2</code> | Match <b>to be or not to be</b>                       |
| <code>([^\s])\1{2}</code>           | Match non-space, then same twice more <b>aaa, ...</b> |
| <code>\b(\w+)\s+\1\b</code>         | Match doubled words                                   |

### Non-capturing group

`on(?:click\\|load)`

Faster than:  
`on(click\\|load)`

Use non-capturing or atomic groups when possible

### Atomic groups

`(?>red\\|green\\|blue)`

Faster than non-capturing

`(?>id\\|identity)\\b`

Match `id`, but not `identity`

"id" matches, but `\\b` fails after atomic group, parser doesn't backtrack into group to retry 'identity'

If alternatives overlap, order longer to shorter.

### Lookaround

`(?= )`

Lookahead, if you can find ahead

`(?! )`

Lookahead, if you can not find ahead

`(?<= )`

Lookbehind, if you can find behind

`(?<!= )`

Lookbehind, if you can NOT find behind

`\\b\\w+?(?=ing\\b)`

Match `warbling`, `string`, `fishing`, ...

`\\b(?!\\w+ing\\b)\\w+\\b`

Words NOT ending in "ing"

`(?<=\\bpre).*?\\b`

Match `pretend`, `present`, `prefix`, ...

`\\b\\w{3}(?<!=pre)\\w*?\\b`

Words NOT starting with "pre"

`\\b\\w+(?<!=ing)\\b`

Match words NOT ending in "ing"

### If-then-else

Match "Mr." or "Ms." if word "her" is later in string

`M(?:?=.*?\\bher\\b)s|r)\\.`

requires lookaround for IF condition

# # RegEx in Python

Getting started

Import the regular expressions module

```
import re
```

Examples

re.search()

```
>>> sentence = 'This is a sample string'
>>> bool(re.search(r'this', sentence, flags=re.I))
True
>>> bool(re.search(r'xyz', sentence))
False
```

re.findall()

```
>>> re.findall(r'\bs?pare?\b', 'par spar apparent spare part pare')
['par', 'spar', 'spare', 'pare']
>>> re.findall(r'\b0*[1-9]\d{2,}\b', '0501 035 154 12 26 98234')
['0501', '154', '98234']
```

re.finditer()

```
>>> m_iter = re.finditer(r'[0-9]+', '45 349 651 593 4 204')
>>> [m[0] for m in m_iter if int(m[0]) < 350]
['45', '349', '4', '204']
```

re.split()

```
>>> re.split(r'\d+', 'Sample123string42with777numbers')
['Sample', 'string', 'with', 'numbers']
```

re.sub()

```
>>> ip_lines = "catapults\nconcatenate\ncat"
>>> print(re.sub(r'^', r'* ', ip_lines, flags=re.M))
* catapults
* concatenate
* cat
```

re.compile()

```
>>> pet = re.compile(r'dog')
>>> type(pet)
<class '_sre.SRE_Pattern'>
>>> bool(pet.search('They bought a dog'))
True
>>> bool(pet.search('A cat crossed their path'))
False
```

#### Functions

|                          |                                                                   |
|--------------------------|-------------------------------------------------------------------|
| <code>re.findall</code>  | Returns a list containing all matches                             |
| <code>re.finditer</code> | Return an iterable of match objects (one for each match)          |
| <code>re.search</code>   | Returns a Match object if there is a match anywhere in the string |
| <code>re.split</code>    | Returns a list where the string has been split at each match      |
| <code>re.sub</code>      | Replaces one or many matches with a string                        |
| <code>re.compile</code>  | Compile a regular expression pattern for later use                |
| <code>re.escape</code>   | Return string with all non-alphanumerics backslashed              |

#### Flags

|                   |                            |                                                    |
|-------------------|----------------------------|----------------------------------------------------|
| <code>re.I</code> | <code>re.IGNORECASE</code> | Ignore case                                        |
| <code>re.M</code> | <code>re.MULTILINE</code>  | Multiline                                          |
| <code>re.L</code> | <code>re.LOCALE</code>     | Make <code>\w, \b, \s</code> locale dependent      |
| <code>re.S</code> | <code>re.DOTALL</code>     | Dot matches all (including newline)                |
| <code>re.U</code> | <code>re.UNICODE</code>    | Make <code>\w, \b, \d, \s</code> unicode dependent |
| <code>re.X</code> | <code>re.VERBOSE</code>    | Readable style                                     |

## # Regex in JavaScript

#### test()

```
let textA = 'I like APPlEs very much';
```

```
let textB = 'I like APPles';
let regex = /apples$/i

// Output: false
console.log(regex.test(textA));

// Output: true
console.log(regex.test(textB));
```

search()

```
let text = 'I like APPles very much';
let regexA = /apples/;
let regexB = /apples/i;

// Output: -1
console.log(text.search(regexA));

// Output: 7
console.log(text.search(regexB));
```

exec()

```
let text = 'Do you like apples?';
let regex= /apples/;

// Output: apples
console.log(regex.exec(text)[0]);

// Output: Do you like apples?
console.log(regex.exec(text).input);
```

match()

```
let text = 'Here are apples and apPleS';
let regex = /apples/gi;

// Output: [ "apples", "apPleS" ]
console.log(text.match(regex));
```

split()

```
let text = 'This 593 string will be brok294en at places where d1gits are.';
let regex = /\d+/g
```

```
// Output: [ "This ", " string will be brok", "en at places where d", "gits are."
console.log(text.split(regex))
```

matchAll()

```
let regex = /t(e)(st(\d?))/g;
let text = 'test1test2';
let array = [...text.matchAll(regex)];

// Output: ["test1", "e", "st1", "1"]
console.log(array[0]);

// Output: ["test2", "e", "st2", "2"]
console.log(array[1]);
```

replace()

```
let text = 'Do you like aPPles?';
let regex = /apples/i

// Output: Do you like mangoes?
let result = text.replace(regex, 'mangoes');
console.log(result);
```

replaceAll()

```
let regex = /apples/gi;
let text = 'Here are apples and apPleS';

// Output: Here are mangoes and mangoes
let result = text.replaceAll(regex, "mangoes");
console.log(result);
```

## # Regex in PHP

Functions

`preg_match()`

Performs a regex match

`preg_match_all()`

Perform a global regular expression match



|                                      |                                                                  |
|--------------------------------------|------------------------------------------------------------------|
| <code>preg_replace_callback()</code> | Perform a regular expression search and replace using a callback |
| <code>preg_replace()</code>          | Perform a regular expression search and replace                  |
| <code>preg_split()</code>            | Splits a string by regex pattern                                 |
| <code>preg_grep()</code>             | Returns array entries that match a pattern                       |

#### preg\_replace

```
$str = "Visit Microsoft!";
$regex = "/microsoft/i";

// Output: Visit QuickRef!
echo preg_replace($regex, "QuickRef", $str);
```

#### preg\_match

```
$str = "Visit QuickRef";
$regex = "#quickref#i";

// Output: 1
echo preg_match($regex, $str);
```

#### preg\_matchall

```
$regex = "/[a-zA-Z]+ (\d+)/";
$input_str = "June 24, August 13, and December 30";
if (preg_match_all($regex, $input_str, $matches_out)) {

    // Output: 2
    echo count($matches_out);

    // Output: 3
    echo count($matches_out[0]);

    // Output: Array("June 24", "August 13", "December 30")
    print_r($matches_out[0]);

    // Output: Array("24", "13", "30")
    print_r($matches_out[1]);
}
```

#### preg\_grep

```
$arr = ["Jane", "jane", "Joan", "JANE"];  
$regex = "/Jane/";
```

```
// Output: Jane  
echo preg_grep($regex, $arr);
```

preg\_split

```
$str = "Jane\tKate\nLucy Marion";  
$regex = "@\s@";
```

```
// Output: Array("Jane", "Kate", "Lucy", "Marion")  
print_r(preg_split($regex, $str));
```

## # Regex in Java

Styles

First way

```
Pattern p = Pattern.compile(".s", Pattern.CASE_INSENSITIVE);  
Matcher m = p.matcher("aS");  
boolean s1 = m.matches();  
System.out.println(s1); // Outputs: true
```

Second way

```
boolean s2 = Pattern.compile("[0-9]+").matcher("123").matches();  
System.out.println(s2); // Outputs: true
```

Third way

```
boolean s3 = Pattern.matches(".s", "XXXX");  
System.out.println(s3); // Outputs: false
```

Pattern Fields

CANON\_EQ

Canonical equivalence

CASE\_INSENSITIVE

Case-insensitive matching

COMMENTS

Permits whitespace and comments

DOTALL

Dottall mode

MULTILINE

Multiline mode

UNICODE\_CASE

Unicode-aware case folding

UNIX\_LINES

Unix lines mode

## Methods

## Pattern

- Pattern compile(String regex [, int flags])
- boolean matches([String regex, ] CharSequence input)
- String[] split(String regex [, int limit])
- String quote(String s)

## Matcher

- int start([int group | String name])
- int end([int group | String name])
- boolean find([int start])
- String group([int group | String name])
- Matcher reset()

## String

- boolean matches(String regex)
- String replaceAll(String regex, String replacement)
- String[] split(String regex[, int limit])

There are more methods ...

## Examples

Replace sentence:

```
String regex = "[A-Z\\n]{5}$";  
String str = "I like APP\\nLE";
```

```

Pattern p = Pattern.compile(regex, Pattern.MULTILINE);
Matcher m = p.matcher(str);

// Outputs: I like Apple!
System.out.println(m.replaceAll("pple!"));

```

Array of all matches:

```

String str = "She sells seashells by the Seashore";
String regex = "\\w*se\\w*";

Pattern p = Pattern.compile(regex, Pattern.CASE_INSENSITIVE);
Matcher m = p.matcher(str);

List<String> matches = new ArrayList<>();
while (m.find()) {
    matches.add(m.group());
}

// Outputs: [sells, seashells, Seashore]
System.out.println(matches);

```

## # Regex in MySQL

### Functions

|                  |                                                                       |
|------------------|-----------------------------------------------------------------------|
| REGEXP           | Whether string matches regex                                          |
| REGEXP_INSTR()   | Starting index of substring matching regex<br>(NOTE: Only MySQL 8.0+) |
| REGEXP_LIKE()    | Whether string matches regex<br>(NOTE: Only MySQL 8.0+)               |
| REGEXP_REPLACE() | Replace substrings matching regex<br>(NOTE: Only MySQL 8.0+)          |
| REGEXP_SUBSTR()  | Return substring matching regex<br>(NOTE: Only MySQL 8.0+)            |

### REGEXP

expr REGEXP pat

#### Examples

```
mysql> SELECT 'abc' REGEXP '^[a-d]';
1
mysql> SELECT name FROM cities WHERE name REGEXP '^A';
mysql> SELECT name FROM cities WHERE name NOT REGEXP '^A';
mysql> SELECT name FROM cities WHERE name REGEXP 'A|B|R';
mysql> SELECT 'a' REGEXP 'A', 'a' REGEXP BINARY 'A';
1 0
```

#### REGEXP\_REPLACE

REGEXP\_REPLACE(expr, pat, repl[, pos[, occurrence[, match\_type]])

#### Examples

```
mysql> SELECT REGEXP_REPLACE('a b c', 'b', 'X');
a X c
mysql> SELECT REGEXP_REPLACE('abc ghi', '[a-z]+', 'X', 1, 2);
abc X
```

#### REGEXP\_SUBSTR

REGEXP\_SUBSTR(expr, pat[, pos[, occurrence[, match\_type]])

#### Examples

```
mysql> SELECT REGEXP_SUBSTR('abc def ghi', '[a-z]+');
abc
mysql> SELECT REGEXP_SUBSTR('abc def ghi', '[a-z]+', 1, 3);
ghi
```

#### REGEXP\_LIKE

REGEXP\_LIKE(expr, pat[, match\_type])

#### Examples

```
mysql> SELECT regexp_like('aba', 'b+')
1
mysql> SELECT regexp_like('aba', 'b{2}')
0
mysql> # i: case-insensitive
```

```
mysql> SELECT regexp_like('Abba', 'ABBA', 'i');
1
mysql> # m: multi-line
mysql> SELECT regexp_like('a\nb\nc', '^b$', 'm');
1
```

## REGEXP\_INSTR

REGEXP\_INSTR(expr, pat[, pos[, occurrence[, return\_option[, match\_type]]]])

### Examples

```
mysql> SELECT regexp_instr('aa aaa aaaa', 'a{3}');
2
mysql> SELECT regexp_instr('abba', 'b{2}', 2);
2
mysql> SELECT regexp_instr('abbabba', 'b{2}', 1, 2);
5
mysql> SELECT regexp_instr('abbabba', 'b{2}', 1, 3, 1);
7
```

## Related Cheatsheet

### Grep Cheatsheet

[Quick Reference](#)

## Recent Cheatsheet

### Remote Work Revolution Cheatsheet

[Quick Reference](#)

### Homebrew Cheatsheet

[Quick Reference](#)

### PyTorch Cheatsheet

[Quick Reference](#)

### Taskset Cheatsheet

[Quick Reference](#)